

Phase 4 — Human-in-the-Loop (HITL)

Roadmap position: Unit 08 — "Human-in-the-loop" **Phase:** 4 of 6 (Foundation → Control Flow → Persistence & Memory → **Human-in-the-Loop** → Multi-Agent & Tools → Production) **Difficulty:** Advanced (but explained from zero assumed knowledge) **Stack used in every lab:** Node.js + TypeScript, `@langchain/langgraph`, `@langchain/openai` (OpenAI API for any LLM calls)

Table of Contents

1. Before You Start — Setup
2. Why Human-in-the-Loop Exists
3. Topic 1 — `interrupt()` : Dynamically Pausing Execution
4. Topic 2 — Static Breakpoints: `interruptBefore` / `interruptAfter`
5. Topic 3 — Resuming a Paused Graph with `Command(resume=...)`
6. Topic 4 — Approval Workflows for Sensitive Actions
7. Topic 5 — Editing State During a Pause (Human Correction)
8. Phase 4 Final Project — Human-Approved Email Assistant
9. Phase 4 Master Knowledge Check
10. Glossary (Phase 4 terms)

0. Before You Start — Setup

Before touching any HITL concepts, get a working project. We'll reuse this exact project for every lab in this phase — just add new files.

Why this setup matters

LangGraph's human-in-the-loop features **only work with a checkpointer**. A checkpointer is the component that saves the graph's state to memory/disk every time it pauses, so that when a human responds hours (or days) later, the graph can pick up exactly where it left off. If you skip this, "pausing" is impossible — the graph would just lose all its progress. Think of a checkpointer like a **video game's save file**: without it, "pause and resume later" doesn't exist, you'd have to start the level over.

Step-by-step setup

```
mkdir langgraph-hitl-phase4
cd langgraph-hitl-phase4
npm init -y
npm install @langchain/langgraph @langchain/openai @langchain/core zod dotenv
npm install -D typescript tsx @types/node
npx tsc --init
```

What each command does:

Command	What it does	Why we need it
---------	--------------	----------------

<code>npm init -y</code>	Creates a <code>package.json</code> with default values	Every Node project needs this manifest file to track dependencies
<code>npm install @langchain/langgraph</code>	Installs LangGraph.js — the graph engine	This is the core library this entire roadmap is about
<code>npm install @langchain/openai</code>	Installs the OpenAI chat-model wrapper	Lets us call GPT models from inside graph nodes
<code>npm install @langchain/core</code>	Installs shared LangChain primitives (messages, runnables)	<code>@langchain/langgraph</code> depends on these types
<code>npm install zod</code>	Installs a schema-validation library	We use it to define our graph's <code>State</code> shape with type safety
<code>npm install -D typescript tsx @types/node</code>	Dev tools: TypeScript compiler, a TS runner, Node type definitions	<code>tsx</code> lets us run <code>.ts</code> files directly (<code>npx tsx file.ts</code>) without a manual build step
<code>npx tsc --init</code>	Generates a <code>tsconfig.json</code>	Configures how TypeScript compiles your code

Open the generated `tsconfig.json` and make sure these are set (edit if needed):

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "esModuleInterop": true,
    "strict": true,
    "skipLibCheck": true
  }
}
```

Create a `.env` file in your project root:

```
OPENAI_API_KEY=sk-your-key-here
```

Why a `.env` file? Hardcoding API keys directly in your code is a security risk — if you ever push your code to GitHub, your key is exposed to the world (and bots will find and abuse it within minutes).

`.env` keeps secrets out of your source code. Add `.env` to a `.gitignore` file too.

Verify your setup works

Create `00-check.ts`:

```
import "dotenv/config";
import { ChatOpenAI } from "@langchain/openai";

async function main() {
  const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });
  const response = await model.invoke("Say 'setup works' and nothing else.");
  console.log(response.content);
}

main();
```

Run it:

```
npx tsx 00-check.ts
```

Expected result: the console prints `setup works` (or very close to it).

How to verify it worked: If you see text output and no red error stack trace, your API key and packages are correctly wired up. If you get an `Incorrect API key` error, double check your `.env` file

has no extra spaces or quotes around the key.

Common beginner mistakes: - Forgetting `import "dotenv/config";` at the very top — without it, `process.env.OPENAI_API_KEY` is `undefined` and you'll get an auth error. - Putting the `.env` file in the wrong folder (it must sit next to `package.json`, in the project root). - Using a model name that doesn't exist (typos like `gpt-4o-min`) — you'll get a "model not found" error.

1. Why Human-in-the-Loop Exists

The problem in plain English

Imagine you built an AI assistant that can read your email and **send replies on your behalf**. Fully autonomous AI agents are powerful, but they're also a little scary in high-stakes situations:

- What if the agent misreads the situation and sends an angry email to your boss?
- What if it's about to charge a customer's credit card the wrong amount?
- What if it's going to delete a database table?

In all of these cases, you don't want to ban the AI from ever doing these actions — you just want a **human to glance at it first** and say "yes, go ahead" or "no, stop." That's the entire idea behind Human-in-the-Loop (HITL).

Analogy: the bank wire transfer

Think about how banks handle large wire transfers. A teller (or an automated system) prepares the transfer, but for amounts over a certain threshold, the transaction doesn't go through automatically — it sits in a queue waiting for a manager's approval. The system isn't redesigned from scratch; it simply **pauses at one specific point** and waits for a signal before continuing. That pause-and-resume mechanic is exactly what LangGraph's HITL tools give you, except the "teller" is your AI agent and the "manager" is you (or any human reviewer) using your application's UI.

Why earlier phases couldn't do this

In Phase 1-2 you learned that a LangGraph graph is just a state machine made of nodes and edges, and in Phase 3 you learned about checkpointers, which let a graph save and reload its state. HITL is the natural next step: **because the graph's state can already be saved to a checkpoint, LangGraph can literally stop running mid-graph, return control to your application, and wait — possibly for milliseconds, possibly for days — until a human provides an answer.** Nothing is lost while it waits, because the checkpointer remembers everything.

The four building blocks of HITL (what this phase teaches)

1. `interrupt()` — a function you call *inside a node* to dynamically pause and ask the human something, then continue from that exact point once they answer.
2. **Static breakpoints** (`interruptBefore` / `interruptAfter`) — a way to tell the *compiled graph* "always pause right before/after this specific node," without writing any pause logic inside the node itself.
3. `Command(resume=...)` — how your application tells a paused graph "here's the human's answer, please continue."
4. **Approval workflows** — the real-world *pattern* that combines the three tools above: pause before a risky action, show it to a human, let them approve/reject/edit, then continue or reroute accordingly.

When to use HITL (and when not to)

Use HITL when...	Don't bother with HITL when...
The action is irreversible (sending money, sending an email, deleting data)	The action is read-only (searching, summarizing, answering a question)
The AI's confidence/accuracy is genuinely uncertain	The task is simple, low-stakes, and well-tested
Regulations or company policy require sign-off	You're prototyping and want fast iteration without manual clicks
You want a human to fill in missing information the AI can't know	All the information is already available in state

A mental model to hold onto

Picture your graph as a hallway with rooms (nodes) connected by doors (edges). Normally, execution walks from room to room without stopping. HITL adds a **locked door** at certain points. The graph can knock — that's `interrupt()` — and it will wait at that locked door, doing absolutely nothing, burning no compute, until someone on the other side (your application, controlled by a human) turns the key and says "continue" — that's `Command(resume=...)`. Crucially, **the graph doesn't restart from room one when it resumes** — it picks up in the exact same room, with all the furniture (state) exactly as it was left.

Knowledge Check — Section 1

Summary: Human-in-the-loop lets a LangGraph graph pause at a deliberate point, hand control back to a human for review or input, and then resume from that exact spot — all powered by the checkpointing's ability to persist state. It exists because fully autonomous agents are risky for irreversible or high-stakes actions.

Key takeaways: - HITL requires a checkpointer — no checkpointer, no pausing. - The graph genuinely freezes; no computation happens while paused. - There are two ways to pause: dynamically from inside a node (`interrupt()`) or statically when compiling the graph (`interruptBefore / interruptAfter`). - Resuming is done by passing `Command(resume=...)` back into the graph.

Practice questions: 1. Why can't a graph pause without a checkpointer attached? 2. Give two real-world examples (outside of email/banking) where you'd want a human approval step before an AI agent acts. 3. What's the difference between an action being "irreversible" vs. "low-stakes" — why does that distinction matter for deciding whether to add HITL?

Mini exercise: Write (in plain English, no code yet) a one-paragraph description of an agent you'd want to build that needs a human approval step, identifying exactly *which single action* in that agent's flow should be gated by HITL.

2. Topic 1 — `interrupt()` : Dynamically Pausing Execution

2.1 Concept and Theory

What is `interrupt()` ?

`interrupt()` is a function you import from `@langchain/langgraph` and call **from inside a node's function body**. When a node calls `interrupt(someValue)`, three things happen:

1. The graph **immediately stops executing** at that exact line.
2. The `someValue` you passed in is sent back out to whatever code called `graph.invoke()` (your application), packaged inside the result as an "interrupt" payload.
3. The graph's state — everything computed so far, including the partial work of the *current*

node, up to that point — is preserved by the checkpointer.

Your application can now show that payload to a human (e.g., render it as a question in a chat UI), wait for their response, and then call the graph again with `Command(resume=humanAnswer)`. The node that called `interrupt()` will **resume from that exact line**, and `interrupt()` itself will behave as if it *returned* the human's answer — like waking up from a pause with the answer already in hand.

Why does it exist, conceptually?

Without `interrupt()`, the only way to "ask a question mid-task" would be to manually split your node into two separate nodes and write custom routing logic to simulate a pause. `interrupt()` removes that complexity: you can write a node as one continuous piece of logic — "do step A, ask a question, do step B based on the answer" — and let LangGraph handle the pausing/resuming machinery for you under the hood.

Analogy: a phone call placed on hold

Imagine you're a customer service rep mid-conversation with a customer, and you need to check something with your manager. You don't hang up the phone and call the customer back from scratch — you say "please hold," walk over, ask your manager, walk back, and continue the *same* conversation exactly where you left off. `interrupt()` is "please hold." The conversation's context (the call) isn't lost; it's just paused.

How it actually works under the hood

This is the part beginners often find confusing, so let's slow down:

- When `interrupt()` is called, LangGraph throws a special internal signal that **unwinds execution out of the node and out of the graph**, back to your `invoke()` / `stream()` call.
- Your call to `graph.invoke()` returns *without error* — instead, the special interrupt information is attached to the result (or surfaced via the `stream()` event of type `"__interrupt__"`).
- When you later call `graph.invoke(new Command({ resume: humanAnswer }), config)` using the **same** `thread_id`, LangGraph reloads the last checkpoint and **re-runs the node from the beginning of its function body** — but this time, every `interrupt()` call that has *already been answered* (in execution order) immediately returns the previously-given answer instead of pausing again.

This last point is critical: **the node function re-runs from the top**, not from the middle. This means any code in the node *before* the `interrupt()` call will run again on resume. For most nodes this is harmless (e.g., re-reading values from state), but it means you should avoid doing things with real side effects (like calling a paid API or sending an email) *before* an `interrupt()` call inside the same node — because that side effect would happen again every time the node re-runs after a resume. If you need multiple interrupts in one node, or side effects mixed with interrupts, structure your node carefully (we'll practice this in the lab).

Syntax

```
import { interrupt } from "@langchain/langgraph";

async function myNode(state: typeof State.State) {
  const humanAnswer = interrupt({
    question: "Should I proceed with this refund?",
    amount: state.refundAmount,
  });
  // Execution PAUSES here until resumed.
  // Once resumed, `humanAnswer` holds whatever value was passed to Command({ resume: ... })

  return { approved: humanAnswer === "yes" };
}
```

When to use `interrupt()` vs. static breakpoints (preview)

`interrupt()` is best when the *decision to pause* depends on **runtime data** — e.g., "only pause if the refund amount is over \$500." Static breakpoints (next topic) are best when you always want to pause at the same fixed point in the graph, regardless of the data. We'll compare them directly in Topic 2.

2.2 Practical Lab — Asking a Clarifying Question Mid-Execution

Goal: Build a tiny LangGraph agent that drafts a birthday message, but if it doesn't know the recipient's name, it pauses and asks you for it using `interrupt()` — then finishes the draft once you reply.

Step 1 — Define the state schema

Create a file called `01-interrupt-basic.ts`.

```
import "dotenv/config";
import { StateGraph, Annotation, START, END, MemorySaver, interrupt, Command } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";

const State = Annotation.Root({
  recipientName: Annotation<string>({
    reducer: (_old, next) => next,
    default: () => "",
  }),
  draft: Annotation<string>({
    reducer: (_old, next) => next,
    default: () => "",
  }),
});
```

Why we do this: This is the same `Annotation.Root` pattern from Phase 1 (State & Schema). We need two fields: `recipientName` (which might start empty and get filled in by a human) and `draft` (the message we generate). The `reducer: (_old, next) => next` simply means "always overwrite with the newest value" — the simplest possible reducer, appropriate here because we're not merging lists or accumulating anything.

Step 2 — Write the node that conditionally interrupts

```
const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0.7 });

async function draftMessageNode(state: typeof State.State) {
  let name = state.recipientName;

  if (!name) {
    // No name in state yet — pause and ask a human for it.
    name = interrupt({
      question: "I don't have the recipient's name. Who is this birthday message for?",
    });
  }

  const response = await model.invoke(
    `Write a short, warm, two-sentence birthday message for someone named ${name}.`
  );

  return { recipientName: name, draft: response.content as string };
}
```

Why we do this: - We check `if (!name)` — this means the `interrupt()` call only happens *conditionally*, based on runtime state. That's the key advantage of `interrupt()` over a static breakpoint: the pause is *data-driven*. - We call `interrupt({ question: ... })` with an object, not a plain string. **Why an object?** In real applications you'll often want to pass structured information (a

question, plus context, plus maybe a list of valid options) so your UI can render something richer than plain text. The payload can be any JSON-serializable value. - Notice the LLM call (`model.invoke(...)`) happens *after* the `interrupt()`. This is intentional — remember from the theory section that code *before* an `interrupt()` re-runs on resume. By putting the (somewhat costly) LLM call *after* the `interrupt`, we avoid calling the LLM twice.

Step 3 — Build and compile the graph with a checkpointer

```
const graph = new StateGraph(State)
  .addNode("draftMessage", draftMessageNode)
  .addEdge(START, "draftMessage")
  .addEdge("draftMessage", END)
  .compile({ checkpointer: new MemorySaver() });
```

Why we do this: Just like in Phase 3, the `checkpointer` is **mandatory** for any HITL feature to work. `MemorySaver` is fine for this local lab (it's in-memory and disappears when the script ends); in Topic 5 and the Final Project we'll discuss when you'd want a persistent one like `SqliteSaver` instead.

Step 4 — Run it and observe the interrupt

```
async function main() {
  const config = { configurable: { thread_id: "birthday-1" } };

  // First call — no name provided, so it should interrupt.
  const result1 = await graph.invoke({ recipientName: "", draft: "" }, config);
  console.log("--- First invoke result ---");
  console.log(JSON.stringify(result1, null, 2));
}

main();
```

Run it:

```
npx tsx 01-interrupt-basic.ts
```

Expected result: Instead of a normal `{ recipientName, draft }` object, you should see a result that includes an `__interrupt__` field, something like:

```
{
  "recipientName": "",
  "draft": "",
  "__interrupt__": [
    {
      "value": { "question": "I don't have the recipient's name. Who is this birthday message for?" },
      "resumable": true,
      ...
    }
  ]
}
```

How to verify it worked: The presence of `__interrupt__` in the output, and the absence of a generated `draft`, confirms execution paused exactly where we expected — *before* the LLM call ran. If you instead see a filled-in `draft`, something is wrong (check that your `if (!name)` block runs before the `model.invoke()` call).

Step 5 — Resume the graph with the human's answer

Add this to the bottom of `main()`, replacing the single `console.log`:

```
async function main() {
  const config = { configurable: { thread_id: "birthday-1" } };
  ...
  // ...
}
```

```
const result1 = await graph.invoke({ recipientName: "", draft: "" }, config);
console.log("--- Paused. Interrupt payload: ---");
console.log(result1.__interrupt__);

// Simulate a human answering the question.
const result2 = await graph.invoke(new Command({ resume: "Priya" }), config);
console.log("--- Resumed. Final result: ---");
console.log(result2);
}

main();
```

Run it again:

```
npx tsx 01-interrupt-basic.ts
```

Expected result: The second log should print something like:

```
{ "recipientName": "Priya", "draft": "Happy birthday, Priya! Wishing you a day as wonderful as you are." }
```

Why this works: Because we reused the **same thread_id ("birthday-1")** for both calls, LangGraph loaded the exact checkpoint where execution paused, re-ran `draftMessageNode` from the top, hit the `interrupt()` call again — but this time it immediately returned `"Priya"` (the value we passed via `Command({ resume: "Priya" })`) instead of pausing again. Execution then continued normally into the LLM call.

Common beginner mistakes:

- **Using a different thread_id on resume.** If you change the `thread_id` between the two calls, LangGraph has no paused checkpoint to find, and it will start a brand-new run from scratch (and may interrupt again, or behave unexpectedly).
- **Forgetting to wrap the resume value in `Command({ resume: ... })`.** Calling `graph.invoke("Priya", config)` directly does *not* resume a paused thread — it tries to start a *new* run using `"Priya"` as the initial state, which will likely throw a validation error since it doesn't match your schema.
- **Expecting `interrupt()` to "return" synchronously the first time.** On the *first* run, `interrupt()` never returns at all — it throws control out of the function entirely. Beginners sometimes write code after `interrupt()` expecting it to run immediately and get confused when it doesn't (until they resume).
- **Calling the LLM before the interrupt and not noticing it's now called twice.** If you swap the order in Step 2 (LLM call first, then `interrupt()`), the LLM call will silently run again on every resume — wasting API calls. Always think about what's *before* vs. *after* your `interrupt()` call.

2.3 Knowledge Check — Topic 1

Summary: `interrupt()` is a function called inside a node to dynamically pause graph execution, send a payload to your application, and later resume from that exact point using `Command({ resume: ... })`, with the node function re-running from its start on resume.

Key takeaways:

- `interrupt()` requires a checkpoint to be compiled into the graph.
- It pauses *conditionally*, based on logic you write — unlike static breakpoints.
- The node re-executes from the top on resume; only the `interrupt()` line itself "remembers" its answer.
- Side effects (LLM calls, API calls, sending data) should generally be placed *after* the `interrupt()` call to avoid duplicate execution.
- You must reuse the same `thread_id` for the pause and the resume to be linked.

Practice questions:

1. What value does `interrupt()` "return" the very first time it's called, before any resume happens?
2. Why is it risky to place a paid API call *before* an `interrupt()` call within the same node?
3. What happens if you call `graph.invoke()` with a new `thread_id` after a previous thread paused — does it resume the old one?

Exercise: Modify the lab so that `draftMessageNode` *also* asks (via a second `interrupt()`) whether the tone should be "funny" or "heartfelt," and incorporates that into the prompt sent to the LLM. Make sure both interrupts happen *before* the LLM call.

3. Topic 2 — Static Breakpoints: `interruptBefore` / `interruptAfter`

3.1 Concept and Theory

What are static breakpoints?

Static breakpoints are a *compile-time* configuration that tells the graph: **"Always pause before (or after) this specific node runs, no matter what."** Unlike `interrupt()`, there's no conditional logic involved and no code added inside the node itself — you simply list node names when calling `.compile()`.

```
const graph = workflow.compile({
  checkpointer: new MemorySaver(),
  interruptBefore: ["sendEmail"],
});
```

This means: every single time the graph is about to run the `sendEmail` node, it stops *unconditionally*, before that node executes at all, and waits for a resume signal.

Why does this exist, given that `interrupt()` already exists?

They solve overlapping but distinct problems:

	<code>interrupt()</code>	<code>interruptBefore</code> / <code>interruptAfter</code>
Where defined	Inside the node's code	At <code>.compile()</code> time, outside the node
Pause condition	Conditional / dynamic (your <code>if</code> logic)	Unconditional — always pauses at that node
Payload sent to human	Custom — whatever you pass into <code>interrupt(...)</code>	None automatically — you inspect <code>getState()</code> yourself
Best for	"Ask a question only when needed," "fill in missing info," multi-step Q&A within one node	"Always require sign-off before this node," debugging, simple universal gates
Node code complexity	Slightly higher (you write the pause logic)	Zero — the node doesn't know it's being paused around

Analogy: airport security vs. a doctor's judgment call

`interruptBefore` is like an **airport security checkpoint** — everyone, no exceptions, stops there and gets checked before boarding, regardless of who they are. `interrupt()` is like a **doctor deciding mid-checkup** whether to order extra tests — it depends on what they observe in *that specific patient*. Both are valid forms of "pause and check," but one is a fixed rule applied universally, and the other is a judgment call baked into the process itself.

`interruptBefore` VS. `interruptAfter`

- `interruptBefore: ["nodeName"]` — pause *before* `nodeName` runs. Useful when you want to review/approve the **inputs** to a node before it acts (e.g., review the email draft before the "send" node fires).
- `interruptAfter: ["nodeName"]` — pause *after* `nodeName` finishes, before moving to the next node. Useful when you want to review the **output** a node just produced (e.g., let a human read what the AI generated before it's shown to an end user).

You can specify multiple node names in either array, and you can use both together.

How resuming works with static breakpoints

It's almost identical to `interrupt()`: you call `graph.invoke(null, config)` (passing `null` as input, since there's nothing new to feed in — just "continue"), or `graph.invoke(new Command({ resume: someValue })), config)` if you want to also pass data in (more on this combination in Topic 4). The graph's checkpoint is loaded, and execution simply proceeds from wherever it stopped.

When to prefer static breakpoints

Use static breakpoints when:

- The same node should **always** require approval, with no exceptions (e.g., any database deletion).
- You want a clean separation between business logic (in the node) and approval policy (at compile time) — useful when different deployments of the same graph need different approval rules (e.g., a "strict" compiled version with breakpoints vs. a "fast" compiled version without, both built from the same node functions).
- You're debugging and just want to pause and inspect state at a certain point, temporarily, without modifying node code.

3.2 Practical Lab — Approving a "Send Email" Node

Goal: Build a 2-node graph (`draftEmail` → `sendEmail`) where the graph *always* pauses before `sendEmail`, regardless of content, and only proceeds once a human manually approves it.

Step 1 — Define state and the two nodes

Create `02-static-breakpoint.ts`.

```
import "dotenv/config";
import { StateGraph, Annotation, START, END, MemorySaver, Command } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";

const State = Annotation.Root({
  topic: Annotation<string>({ reducer: (_, n) => n, default: () => "" }),
  emailDraft: Annotation<string>({ reducer: (_, n) => n, default: () => "" }),
  sent: Annotation<boolean>({ reducer: (_, n) => n, default: () => false }),
});

const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0.5 });

async function draftEmailNode(state: typeof State.State) {
  const response = await model.invoke(
    `Write a short professional email (max 4 sentences) about: ${state.topic}`
  );
  return { emailDraft: response.content as string };
}

async function sendEmailNode(state: typeof State.State) {
  // In a real app, this is where you'd call an email-sending API (e.g. SendGrid, Resend).
  console.log("📧 SENDING EMAIL NOW:\n", state.emailDraft);
  return { sent: true };
}
```

Why we do this: Notice `sendEmailNode` contains **zero pause logic** — it doesn't know or care that a human is approving it. That's the whole point of static breakpoints: the node stays simple and focused purely on its job (sending the email), and the "should we pause here" decision lives entirely outside it, at compile time.

Step 2 — Compile with `interruptBefore`

```
const graph = new StateGraph(State)
  .addNode("draftEmail", draftEmailNode)
  .addNode("sendEmail", sendEmailNode)
  .addEdge(START, "draftEmail")
  .addEdge("draftEmail", "sendEmail")
```

```
.addEdge("sendEmail", END)
.compile({
  checkpointer: new MemorySaver(),
  interruptBefore: ["sendEmail"],
});
```

Why we do this: `interruptBefore: ["sendEmail"]` tells LangGraph: "after `draftEmail` finishes and you're about to enter `sendEmail`, stop and wait — every single time, for every thread."

Step 3 — Run, inspect, and approve manually

```
async function main() {
  const config = { configurable: { thread_id: "email-thread-1" } };

  const result1 = await graph.invoke({ topic: "rescheduling tomorrow's 3pm meeting to 4pm" }, config);
  console.log("--- Graph paused. Current state: ---");

  const snapshot = await graph.getState(config);
  console.log("Next node to run:", snapshot.next);
  console.log("Draft so far:", snapshot.values.emailDraft);

  // --- Imagine a human reads the draft in a UI and clicks "Approve" ---
  console.log("\n👤 Human approved. Resuming...\n");

  const result2 = await graph.invoke(null, config);
  console.log("--- Final state: ---");
  console.log(result2);
}

main();
```

Why we do this: - `graph.getState(config)` is a tool from Phase 3 (State Management & Time Travel) — we reuse it here to **inspect exactly what's waiting to run next** (`snapshot.next` will show `["sendEmail"]`) and to read the draft a human would need to review. - Calling `graph.invoke(null, config)` — passing `null` instead of new input or a `Command` — is the simplest way to say "just continue from where you paused, nothing new to add."

Run it:

```
npx tsx 02-static-breakpoint.ts
```

Expected result: You should see the state snapshot print `Next node to run: [ 'sendEmail' ]` and the draft text, *then* — only after the "Human approved" log line — see `📧 SENDING EMAIL NOW:` followed by the email content.

How to verify it worked: Comment out the `graph.invoke(null, config)` line and re-run — the `📧 SENDING EMAIL NOW` line should *never* print, proving the node truly didn't execute until you explicitly resumed it.

Step 4 — Simulate a rejection (don't resume at all)

Add a small variation to show how rejection works — simply *don't* call the resume `invoke`:

```
async function simulateRejection() {
  const config = { configurable: { thread_id: "email-thread-2" } };
  await graph.invoke({ topic: "firing an underperforming vendor" }, config);

  const snapshot = await graph.getState(config);
  console.log("Draft pending rejection:", snapshot.values.emailDraft);
  console.log("✖ Human rejected — the graph simply stays paused forever. No email is ever sent.");
  // We never call graph.invoke(null, config) again for this thread_id.
}
```

Why this matters: Rejection in the simplest case is just... not resuming. The thread stays paused

indefinitely (or until you decide to delete/expire it). In Topic 4 we'll build a *smarter* rejection that reroutes the graph to revise the draft instead of just freezing it forever.

Common beginner mistakes: - **Forgetting that** `interruptBefore` **pauses *before* the node's side effects, not after.** If you wanted to review the email *after* it claims to be sent (for logging purposes, say), you'd use `interruptAfter` instead. - **Calling** `graph.invoke({...someNewInput}, config)` **to resume**, accidentally passing a full new input object instead of `null` or a `Command`. This can overwrite state fields unexpectedly depending on your reducers — always use `null` (or `Command`) for a "no new input, just continue" resume. - **Listing a node name that doesn't exist** in `interruptBefore / interruptAfter` (e.g., a typo) — LangGraph will not error loudly in all versions, so always double-check your node names match exactly what you used in `.addNode(...)`.

3.3 Knowledge Check — Topic 2

Summary: Static breakpoints (`interruptBefore / interruptAfter`) are configured once at `.compile()` time and unconditionally pause execution immediately before or after a named node, keeping node functions free of any pause-related code. Resuming is done with `graph.invoke(null, config)` (or a `Command`).

Key takeaways: - `interruptBefore` pauses before the node's logic runs; `interruptAfter` pauses after it finishes. - No code changes are needed inside the node itself. - `graph.getState(config)` lets you inspect what's paused and what will run next. - "Rejecting" in the simplest form is just never resuming that thread.

Practice questions: 1. What's the key behavioral difference between `interrupt()` and `interruptBefore`? 2. If you wanted to let a human review an AI-generated summary *after* it's created but *before* it's emailed in a separate downstream node, would you use `interruptBefore` or `interruptAfter`, and on which node? 3. What does `snapshot.next` tell you, and why is it useful before deciding whether to resume?

Mini project: Extend the lab so there are *three* nodes: `draftEmail` → `sendEmail` → `logToCustomerRecord`. Add a breakpoint so a human reviews the draft before sending, **and** a second breakpoint so a human confirms before the action is logged to a permanent customer record. Print `snapshot.next` at each pause to confirm both gates work.

4. Topic 3 — Resuming a Paused Graph with `Command(resume=...)`

4.1 Concept and Theory

You've already used `Command({ resume: ... })` in the labs above — now let's go deeper into exactly what it is and why it's designed the way it is.

What is `Command`?

`Command` is a special object (you met an earlier use of it in Phase 2, "Command & Send API," where it combined a state update *and* a routing decision). In the HITL context, `Command` is the vehicle for **delivering a human's response back into a paused graph**. It supports a few relevant fields:

```
new Command({
  resume: <any value>, // the answer to give back to a waiting interrupt()
  update: <partial state>, // optionally also update state directly (advanced; Topic 5)
  goto: <node name>, // optionally also force routing to a specific node (advanced)
});
```

For basic HITL, you'll mostly use just `resume`.

Why not just call `graph.invoke(humanAnswer, config)` directly?

Because `graph.invoke()` 's first argument normally means **"new initial input to merge into state."** If you passed a raw human answer like `"yes"` there, LangGraph would try to interpret `"yes"` as a full or partial *state object* matching your schema — which would either throw a validation error or silently merge nonsense into your state. Wrapping it in `Command({ resume: "yes" })` makes the intent unambiguous: *"this is an answer to a pending `interrupt()`, not new top-level state."*

What "resume" actually delivers

When a node previously called `interrupt(payload)`, that call is, at the moment of interruption, like a frozen function call waiting for a return value. `Command({ resume: humanAnswer })` is what makes `interrupt(payload)` finally "return" — `humanAnswer` becomes the value that expression evaluates to, exactly as if you'd written:

```
const humanAnswer = /* magically becomes */ "yes";
```

Multiple interrupts in one node, and resume ordering

If a single node calls `interrupt()` more than once (you tried this in the Topic 1 exercise), LangGraph tracks them **in the order they occur** during that node's execution. Resume values are matched to interrupts in that same order on each subsequent run. This is why it's important to keep the logic *before* each `interrupt()` deterministic and side-effect-free — if the order or number of `interrupt()` calls changes between runs (e.g., because of an `if` branch that behaves differently), the resume values could get matched to the wrong question. **Best practice: keep the path leading up to each `interrupt()` call stable and predictable.**

Resuming across process restarts

Because the checkpoint is the single source of truth (not anything held in memory in your running Node.js process), you can technically: 1. Run a script that pauses on `interrupt()` and exits completely. 2. Hours later, start a *brand new* Node.js process. 3. As long as you use a **persistent checkpointer** (like `SqliteSaver` or `PostgresSaver` from Phase 3 — not `MemorySaver`, which is wiped when the process ends) and the same `thread_id`, calling `graph.invoke(new Command({ resume: ... })), config)` from the new process will correctly resume the old, paused run.

This is the property that makes HITL genuinely useful in production: the human doesn't need to respond instantly, and your server doesn't need to stay running and waiting the whole time.

4.2 Practical Lab — Resuming Across a Simulated "Restart"

Goal: Prove to yourself that resuming works even when the "human's answer" comes from a completely separate script execution, using a persistent SQLite checkpointer instead of `MemorySaver`.

Step 1 — Install the SQLite checkpointer package

```
npm install @langchain/langgraph-checkpoint-sqlite
```

Why we do this: `MemorySaver` only lives inside one running process's RAM. To prove a "real" resume across a restart, we need state saved to disk, which `SqliteSaver` provides (just like in Phase 3, Checkpointers).

Step 2 — Write the "pause" script

Create `03a-resume-pause.ts`:

```
import "dotenv/config";
import { StateGraph, Annotation, START, END, interrupt } from "@langchain/langgraph";
import { SqliteSaver } from "@langchain/langgraph-checkpoint-sqlite";
```

```

const State = Annotation.Root({
  question: Annotation<string>({ reducer: (_o, n) => n, default: () => "" }),
  decision: Annotation<string>({ reducer: (_o, n) => n, default: () => "" }),
});

async function approvalNode(state: typeof State.State) {
  const decision = interrupt({ ask: "Approve the $4,200 vendor invoice? (yes/no)" });
  return { decision: decision as string };
}

const checkpointer = SqliteSaver.fromConnString("hitl-demo.db");

const graph = new StateGraph(State)
  .addNode("approval", approvalNode)
  .addEdge(START, "approval")
  .addEdge("approval", END)
  .compile({ checkpointer });

async function main() {
  const config = { configurable: { thread_id: "invoice-9981" } };
  const result = await graph.invoke({ question: "approve invoice?" }, config);
  console.log("Process A paused. Interrupt payload:", result.__interrupt__);
  console.log("Exiting process now. The checkpoint is saved in hitl-demo.db.");
}

main();

```

Run it:

```
npx tsx 03a-resume-pause.ts
```

Expected result: It prints the interrupt payload and exits cleanly. **How to verify:** Check that a new file `hitl-demo.db` now exists in your project folder — that's the SQLite database file holding the paused checkpoint.

Step 3 — Write a *separate* "resume" script

Create `03b-resume-continue.ts`:

```

import "dotenv/config";
import { StateGraph, Annotation, START, END, interrupt, Command } from "@langchain/langgraph";
import { SqliteSaver } from "@langchain/langgraph-checkpoint-sqlite";

// IMPORTANT: the graph definition must be IDENTICAL (same nodes, same schema)
// to the one used in the pause script, since we're reloading its checkpoint.
const State = Annotation.Root({
  question: Annotation<string>({ reducer: (_o, n) => n, default: () => "" }),
  decision: Annotation<string>({ reducer: (_o, n) => n, default: () => "" }),
});

async function approvalNode(state: typeof State.State) {
  const decision = interrupt({ ask: "Approve the $4,200 vendor invoice? (yes/no)" });
  return { decision: decision as string };
}

const checkpointer = SqliteSaver.fromConnString("hitl-demo.db");

const graph = new StateGraph(State)
  .addNode("approval", approvalNode)
  .addEdge(START, "approval")
  .addEdge("approval", END)
  .compile({ checkpointer });

async function main() {
  const config = { configurable: { thread_id: "invoice-9981" } }; // SAME thread_id as before
  const result = await graph.invoke(new Command({ resume: "yes" }), config);
  console.log("Process B resumed successfully:", result);
}

```

```
main();
```

Run it (notice this is a **completely separate process** from Step 2 — no shared memory at all):

```
npx tsx 03b-resume-continue.ts
```

Expected result:

```
Process B resumed successfully: { question: 'approve invoice?', decision: 'yes' }
```

How to verify it worked: The fact that `decision: 'yes'` appears at all — produced by a script that *never set that field itself* — proves the resume correctly reattached to the checkpoint left behind by the *other* process. If you change `thread_id` in script B to something that wasn't used in script A, you'll instead get an error or an entirely fresh run, proving the link is solely based on `thread_id` + the database file.

Common beginner mistakes:

- **Forgetting that the graph structure (nodes, schema) must match** between the pausing process and the resuming process. If you add/remove/rename nodes between the two runs, LangGraph may not know how to safely resume.
- **Using `MemorySaver` and expecting cross-process resume to work** — it won't, because `MemorySaver` only exists in one process's RAM and is gone the moment that process exits.
- **Reusing the same `hitl-demo.db` `thread_id` for unrelated runs** — like reusing a variable name, this causes confusing state bleed between unrelated tasks. Use clearly unique, meaningful `thread_id`s (e.g., prefix with an entity ID: `invoice-9981`, `order-44213`).

4.3 Knowledge Check — Topic 3

Summary: `Command({ resume: value })` is the dedicated way to feed a human's answer back into a graph that's paused on `interrupt()`. It avoids ambiguity with normal state input. Because all pause/resume state lives in the checkpointer (not in process memory), resuming can happen from a totally different process — or even days later — as long as you use a persistent checkpointer and the same `thread_id`.

Key takeaways:

- `Command({ resume: ... })` $\neq$ passing raw input to `invoke()`.
- Multiple `interrupt()` calls in one node resume in call order.
- `MemorySaver` cannot survive process restarts; use `SqliteSaver` / `PostgresSaver` for anything that needs to persist.
- The `thread_id` is the single link between a paused run and its eventual resume.

Practice questions:

1. Why does `Command({ resume: ... })` exist instead of just reusing `graph.invoke(value, config)`?
2. What would happen (conceptually) if process B used a `thread_id` that process A never created?
3. Why must the node/graph structure stay consistent between the script that pauses and the script that resumes?

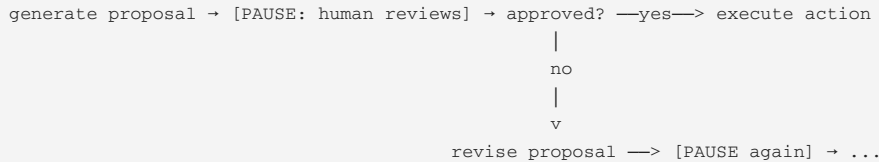
Exercise: Modify `03b-resume-continue.ts` to resume with `"no"` instead of `"yes"`, and add a follow-up `console.log` that prints a different message depending on the `decision` value — simulating how your app might branch its UI based on the human's answer.

5. Topic 4 — Approval Workflows for Sensitive Actions

5.1 Concept and Theory

This topic doesn't introduce brand-new LangGraph APIs — instead, it teaches the **pattern** of combining everything from Topics 1–3 into something genuinely useful: a real approval gate that can *approve*, *reject-and-revise*, or *reject-and-abort* a sensitive action.

The general shape of an approval workflow



This is a **loop with a human in the middle** — structurally similar to the "self-correcting loop" pattern from Phase 2 (Conditional Edges & Branching), except instead of an AI critic deciding whether to loop again, a *human* decides.

Why a conditional edge is the right tool here

After resuming from the pause, we need to **route differently** depending on what the human decided: continue to the sensitive action, or go back and revise. This is exactly what `addConditionalEdges()` (Phase 2) is for. The routing function reads the `decision` field set by the `interrupt()`'s resume value and returns the name of the next node.

Designing the interrupt payload thoughtfully

A good approval-step `interrupt()` payload typically includes: - **What the AI is proposing to do** (e.g., the exact email text, the exact dollar amount). - **Why** (a short rationale, if you have one) — this helps the human trust or distrust the proposal faster. - **What responses are valid** (e.g., `"approve"`, `"reject"`, or `"edit"` with new text) — this is essentially a tiny contract between your graph and your UI.

Three common outcomes to support

1. **Approve** — proceed exactly as proposed.
2. **Reject and abort** — stop the whole workflow; nothing happens.
3. **Reject and revise** — go back and regenerate the proposal, optionally taking human feedback into account (e.g., "make it shorter" or "the amount should be \$300, not \$500").

We'll build all three in the lab below.

5.2 Practical Lab — Build an Approval Gate with Revise/Reject Branching

Goal: Build a graph that proposes a refund amount to a customer, pauses for human approval, and supports three outcomes: approve & process, reject & abort, or reject & revise with feedback (looping back).

Step 1 — Define state

Create `04-approval-workflow.ts`:

```
import "dotenv/config";
import {
  StateGraph, Annotation, START, END, MemorySaver, interrupt, Command,
} from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";

const State = Annotation.Root({
  customerComplaint: Annotation<string>({ reducer: (_, n) => n, default: () => "" }),
  proposedRefund: Annotation<number>({ reducer: (_, n) => n, default: () => 0 }),
  reviewerFeedback: Annotation<string>({ reducer: (_, n) => n, default: () => "" }),
  status: Annotation<"pending" | "approved" | "rejected">({
    reducer: (_, n) => n,
    default: () => "pending",
  }),
});
```



```
});
```

Why we do this: `reviewerFeedback` is new — it stores any free-text notes a human reviewer gives when requesting a revision (e.g., "this refund is too high"), so the regeneration step can actually use that feedback instead of guessing blindly again.

Step 2 — Write the proposal node

```
const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0.3 });

async function proposeRefundNode(state: typeof State.State) {
  const feedbackNote = state.reviewerFeedback
    ? `
    \nA reviewer previously gave this feedback, take it into account: "${state.reviewerFeedback}"
    `
    : "";

  const response = await model.invoke(
    `A customer wrote this complaint: "${state.customerComplaint}".
    Propose a fair refund amount in USD as a single number only (no symbols, no text).${feedbackNote}`
  );

  const amount = parseFloat((response.content as string).replace(/[^0-9.]/g, "")) || 0;
  return { proposedRefund: amount };
}
```

Why we do this: This node is reusable for *both* the first attempt and any later revision — that's why it checks `state.reviewerFeedback` and conditionally includes it in the prompt. This avoids needing two separate "draft" nodes for "first try" vs. "retry."

Step 3 — Write the human approval node

```
async function humanApprovalNode(state: typeof State.State) {
  const response = interrupt({
    action: "approve_refund",
    proposedRefund: state.proposedRefund,
    customerComplaint: state.customerComplaint,
    validResponses: ["approve", "reject_abort", "reject_revise"],
  }) as { decision: string; feedback?: string };

  if (response.decision === "approve") {
    return { status: "approved" as const };
  }
  if (response.decision === "reject_abort") {
    return { status: "rejected" as const };
  }
  // reject_revise
  return { status: "pending" as const, reviewerFeedback: response.feedback ?? "" };
}
```

Why we do this: Instead of the resume value being a plain string, here we resume with a small **object** (`{ decision, feedback }`). This is a deliberate design choice — real approval UIs usually need to send back more than one piece of information (the decision *and* optional notes), and `interrupt() / Command({resume})` happily support any JSON-serializable shape, not just strings.

Step 4 — Write the "execute" and routing logic

```
async function processRefundNode(state: typeof State.State) {
  console.log(`🔄 Processing refund of $${state.proposedRefund} for complaint: "${state.customerComplaint}"`);
  return {};
}

function routeAfterApproval(state: typeof State.State) {
  if (state.status === "approved") return "processRefund";
  if (state.status === "rejected") return END;
  return "proposeRefund"; // pending => loop back for a revision
}
```

```
}
```

Why we do this: This is a textbook conditional-edge routing function (Phase 2) — it inspects state set by the previous node and decides the next destination as a plain string. Routing back to "proposeRefund" is what implements the "reject & revise" loop.

Step 5 — Assemble the graph

```
const graph = new StateGraph(State)
  .addNode("proposeRefund", proposeRefundNode)
  .addNode("humanApproval", humanApprovalNode)
  .addNode("processRefund", processRefundNode)
  .addEdge(START, "proposeRefund")
  .addEdge("proposeRefund", "humanApproval")
  .addConditionalEdges("humanApproval", routeAfterApproval, {
    processRefund: "processRefund",
    proposeRefund: "proposeRefund",
    [END]: END,
  })
  .addEdge("processRefund", END)
  .compile({ checkpoint: new MemorySaver() });
```

Step 6 — Drive all three outcomes

```
async function runApproveScenario() {
  console.log("\n=== SCENARIO 1: Approve ===");
  const config = { configurable: { thread_id: "refund-approve" } };
  await graph.invoke({ customerComplaint: "My package arrived damaged." }, config);
  let snap = await graph.getState(config);
  console.log("Proposed:", snap.values.proposedRefund);

  await graph.invoke(new Command({ resume: { decision: "approve" } }), config);
  snap = await graph.getState(config);
  console.log("Final status:", snap.values.status);
}

async function runRejectAbortScenario() {
  console.log("\n=== SCENARIO 2: Reject & Abort ===");
  const config = { configurable: { thread_id: "refund-abort" } };
  await graph.invoke({ customerComplaint: "Item was one day late." }, config);

  await graph.invoke(new Command({ resume: { decision: "reject_abort" } }), config);
  const snap = await graph.getState(config);
  console.log("Final status:", snap.values.status);
}

async function runReviseScenario() {
  console.log("\n=== SCENARIO 3: Reject & Revise, then Approve ===");
  const config = { configurable: { thread_id: "refund-revise" } };
  await graph.invoke({ customerComplaint: "Product broke after one use, very upset." }, config);
  let snap = await graph.getState(config);
  console.log("First proposal:", snap.values.proposedRefund);

  // Reviewer asks for a lower amount with feedback.
  await graph.invoke(
    new Command({ resume: { decision: "reject_revise", feedback: "Cap refunds at $25 for single-use complaint" } }),
    config
  );
  snap = await graph.getState(config);
  console.log("Revised proposal:", snap.values.proposedRefund, "| next node:", snap.next);

  // Now approve the revised proposal.
  await graph.invoke(new Command({ resume: { decision: "approve" } }), config);
  snap = await graph.getState(config);
  console.log("Final status:", snap.values.status);
}
```

```

async function main() {
  await runApproveScenario();
  await runRejectAbortScenario();
  await runReviseScenario();
}

main();

```

Run it:

```
npx tsx 04-approval-workflow.ts
```

Expected result: Three labeled scenario blocks print in your terminal. Scenario 1 ends with `Final status: approved` and a `Processing refund...` line. Scenario 2 ends with `Final status: rejected` and **no** processing line. Scenario 3 shows two different `proposedRefund` values (the first guess, then a lower revised guess influenced by the feedback) before finally approving.

How to verify it worked: - For Scenario 2, confirm `Processing refund` never printed — proving rejection truly blocks the sensitive action. - For Scenario 3, confirm the second `proposedRefund` is different from (ideally lower than) the first, proving the `reviewerFeedback` was actually incorporated into the regenerated prompt, not ignored.

Common beginner mistakes: - **Forgetting to map every possible routing function output in `addConditionalEdges`'s third argument.** If `routeAfterApproval` can return `"processRefund"`, `"proposeRefund"`, or `END`, *all three* must appear as keys in that mapping object, or LangGraph will throw a routing error for the missing case. - **Resuming with a plain string `"approve"` instead of the object shape `{ decision: "approve" }`** the node expects — this causes `response.decision` to be `undefined`, silently falling through to the "revise" branch. Always make sure your resume payload's *shape* matches exactly what the node destructures. - **Not resetting `reviewerFeedback`** after a successful approval if you reuse the thread for another complaint — stale feedback could leak into unrelated future proposals on the same thread. In production, prefer a fresh `thread_id` per complaint.

5.3 Knowledge Check — Topic 4

Summary: Real approval workflows combine `interrupt()`, conditional edges, and looping to support three outcomes — approve, reject-and-abort, and reject-and-revise — turning a single pause point into a flexible human-AI negotiation loop.

Key takeaways: - The same node can serve both the "first attempt" and "revision" purpose if it conditionally reads feedback from state. - Resume payloads can be structured objects, not just strings — design them to match what your UI realistically needs to send. - `addConditionalEdges` after a `humanApproval` node is what turns a single yes/no into a true branching/looping workflow. - "Reject & abort" routes straight to `END`; "reject & revise" routes back to the proposal node, forming a loop.

Practice questions: 1. Why is it useful for `proposeRefundNode` to handle both the first attempt and revisions, rather than having two separate nodes? 2. What would happen if `addConditionalEdges`'s mapping object was missing the `[END]: END` entry? 3. Design (in words) what fields you'd want in the `interrupt()` payload for an approval gate on a "delete user account" action — what would a human need to see to make a safe decision?

Mini project: Add a maximum-revision-count safeguard: track a `revisionCount` field in state, increment it on every "reject_revise," and force the routing function to go straight to `END` (with a `status` of `"rejected"`) if `revisionCount` exceeds 3 — preventing an endless human-in-the-loop negotiation loop.

6. Topic 5 — Editing State During a Pause (Human Correction)

6.1 Concept and Theory

The problem this solves

So far, our humans have only answered *questions* (yes/no/feedback text) via `interrupt()`'s resume value. But sometimes a human doesn't want to answer a question — they want to **directly edit a piece of state themselves**, like fixing a typo in an AI-drafted email, before it continues. This is "human correction," and LangGraph supports it in two complementary ways:

1. **Via the resume payload itself** — have the human supply a *corrected* version of the content as the `resume` value (e.g., resume with the *edited* email text instead of just `"approve"`), and have the node use that corrected text going forward. This is the approach we'll emphasize, because it composes naturally with everything you've already learned.
2. **Via `updateState()`** (introduced in Phase 3, "State Management & Time Travel") — while a thread is paused, you can call `await graph.updateState(config, { emailDraft: "the human's edited text" })` to directly overwrite a field in the *checkpoint*, **without** going through any node at all, and then resume normally afterward. This is useful when the correction doesn't naturally fit as an `interrupt()`'s return value (e.g., a totally separate admin tool edits the draft before continuing).

Why two ways to do "the same thing"?

They serve different architectural needs:

Approach	When to use
Pass corrected value through <code>Command({ resume: correctedValue })</code>	The same flow that paused is the one collecting the correction (e.g., one approval UI form with a text box)
<code>graph.updateState(config, {...})</code> directly	A different process/tool edits state independently of the original pause (e.g., an admin dashboard lets someone fix something on a stalled thread without "answering" any specific interrupt)

Analogy: editing a document

Think of Google Docs. If you're actively in a conversation with a collaborator's comment thread and you reply with your suggested edit text — that's like passing a correction through `resume`. If instead you just open the document yourself and directly retype a sentence, with no comment thread involved at all — that's like `updateState()`. Both change the document; they just go through different "doors."

A subtlety: where does the corrected value get used?

When you resume with a corrected value, **the node code itself must explicitly use that value** — `interrupt()` doesn't automatically know "this is a correction, please overwrite `emailDraft`." You write the node to say, in effect: "use whatever I get back from `interrupt()` as the new draft," exactly like we did with `recipientName` in Topic 1's lab.

6.2 Practical Lab — Let a Human Edit a Draft Before It's Sent

Goal: Build a graph that drafts a message, pauses for review, and lets a human either approve it as-is or supply fully edited replacement text — which then gets used for the final "send" step.

Step 1 — State and draft node

Create `05-human-edit.ts`:

```
import "dotenv/config";
import {
```

```

    StateGraph, Annotation, START, END, MemorySaver, interrupt, Command,
  } from "@langchain/langgraph";
  import { ChatOpenAI } from "@langchain/openai";

  const State = Annotation.Root({
    topic: Annotation<string>({ reducer: (_o, n) => n, default: () => "" }),
    draft: Annotation<string>({ reducer: (_o, n) => n, default: () => "" }),
    finalText: Annotation<string>({ reducer: (_o, n) => n, default: () => "" }),
  });

  const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0.6 });

  async function draftNode(state: typeof State.State) {
    const response = await model.invoke(
      `Write a 2-sentence LinkedIn announcement post about: ${state.topic}`
    );
    return { draft: response.content as string };
  }

```

Step 2 — The review-and-edit node

```

  async function reviewAndEditNode(state: typeof State.State) {
    const result = interrupt({
      instructions: "Review this draft. Either send it as-is, or provide fully edited replacement text.",
      currentDraft: state.draft,
    }) as { editedText?: string };

    // If the human supplied edited text, use it. Otherwise, keep the original draft.
    const finalText = result.editedText && result.editedText.trim().length > 0
      ? result.editedText
      : state.draft;

    return { finalText };
  }

  async function publishNode(state: typeof State.State) {
    console.log("📢 PUBLISHED:", state.finalText);
    return {};
  }

```

Why we do this: The key line is `result.editedText && result.editedText.trim().length > 0 ? result.editedText : state.draft`. This is the explicit logic that decides "did the human actually provide a correction, or did they just approve as-is?" — exactly the subtlety called out in the theory section: **the node must explicitly handle the corrected value**, LangGraph doesn't do this automatically.

Step 3 — Graph assembly

```

const graph = new StateGraph(State)
  .addNode("draft", draftNode)
  .addNode("reviewAndEdit", reviewAndEditNode)
  .addNode("publish", publishNode)
  .addEdge(START, "draft")
  .addEdge("draft", "reviewAndEdit")
  .addEdge("reviewAndEdit", "publish")
  .addEdge("publish", END)
  .compile({ checkpoint: new MemorySaver() });

```

Step 4 — Scenario A: approve as-is

```

  async function approveAsIs() {
    console.log("\n=== Approve as-is ===");
    const config = { configurable: { thread_id: "post-1" } };
    await graph.invoke({ topic: "our team shipping a new HITL feature" }, config);
    const snap = await graph.getState(config);
    console.log("AI draft was:", snap.values.draft);
  }

```

```

    await graph.invoke(new Command({ resume: {} })), config); // no editedText provided
    const final = await graph.getState(config);
    console.log("Used finalText:", final.values.finalText);
  }
}

```

Step 5 — Scenario B: human edits the text

```

async function humanEdits() {
  console.log("\n=== Human edits the draft ===");
  const config = { configurable: { thread_id: "post-2" } };
  await graph.invoke({ topic: "our team shipping a new HITL feature" }, config);
  const snap = await graph.getState(config);
  console.log("AI draft was:", snap.values.draft);

  await graph.invoke(
    new Command({ resume: { editedText: "We just shipped human-in-the-loop approvals — review AI actions be
    config
  });
  const final = await graph.getState(config);
  console.log("Used finalText:", final.values.finalText);
}

async function main() {
  await approveAsIs();
  await humanEdits();
}

main();

```

Run it:

```
npx tsx 05-human-edit.ts
```

Expected result: In Scenario A, `finalText` matches the original AI `draft` exactly. In Scenario B, `finalText` matches the **human-supplied** sentence, not the AI's original draft, and the `<P> PUBLISHED:` line in `publishNode` reflects the edited version.

How to verify it worked: Compare `snap.values.draft` (the AI's original) against `final.values.finalText` (what actually got used) in both scenarios — they should be identical in Scenario A and different in Scenario B.

Step 6 — Bonus: using `updateState()` for an out-of-band correction

```

async function adminOverride() {
  console.log("\n=== Admin tool edits state directly via updateState() ===");
  const config = { configurable: { thread_id: "post-3" } };
  await graph.invoke({ topic: "our quarterly results" }, config);

  // Imagine a SEPARATE admin dashboard does this, with no knowledge of "interrupt payloads":
  await graph.updateState(config, { draft: "Q3 results: revenue up 18% YoY. Full report linked below." });

  // Now resume normally — reviewAndEditNode will interrupt again with the UPDATED draft.
  const snap = await graph.getState(config);
  console.log("Draft after admin override:", snap.values.draft);

  await graph.invoke(new Command({ resume: {} })), config);
  const final = await graph.getState(config);
  console.log("Used finalText:", final.values.finalText);
}

```

Add `await adminOverride();` to `main()` and re-run. **Expected result:** the printed draft reflects the admin's overridden text, not the original AI draft, proving `updateState()` can rewrite state independently of any `interrupt()` exchange.

Common beginner mistakes: - Assuming `interrupt()` automatically merges the human's edit into state. It does not — you must explicitly write the `result.editedText ? ... : ...` logic yourself. - Calling `updateState()` on a thread that isn't currently paused (e.g., one that already ran to completion) — this can have unexpected effects since there's no pending node waiting to consume the change. Generally reserve `updateState()` for paused threads, matching the use case from Phase 3. - Forgetting to re-fetch `getState()` after `updateState()` to confirm the override actually applied before continuing — silent typos in field names (e.g., `Draft` instead of `draft`) will fail to update anything and TypeScript may not catch it if your editor type isn't strict.

6.3 Knowledge Check — Topic 5

Summary: Humans can correct AI-generated content either by supplying replacement values through a `resume` payload (which the node must explicitly check for and use) or by calling `updateState()` directly on a paused thread to overwrite state without going through any node logic at all.

Key takeaways: - `interrupt()` never auto-applies a correction — your node code decides what "no edit given" vs. "edit given" means. - `updateState()` is for out-of-band corrections, independent of any specific `interrupt()` call. - Always verify a correction took effect with `getState()` before resuming/continuing.

Practice questions: 1. Why doesn't `interrupt()` automatically know that a returned value should replace a draft field? 2. When would `updateState()` be more appropriate than passing a corrected value through `resume`? 3. What bug would occur if `reviewAndEditNode` always used `result.editedText` even when it's `undefined`, without the fallback check?

Exercise: Add a third option to `reviewAndEditNode` — a "regenerate" instruction (no edited text, no approval, just "try again with this style note: ___") that routes back to `draftNode` with a style hint stored in state, similar to the "reject & revise" loop from Topic 4.

7. Phase 4 Final Project — Human-Approved Email Assistant

7.1 Project Brief

You will now combine **everything from this phase** — `interrupt()`, static breakpoints, `Command(resume=...)`, full approval workflows, and human edits — into one cohesive, realistic application: an **AI email assistant** that drafts replies to incoming customer messages, and requires human sign-off before anything goes out, with support for approving, editing, or rejecting-and-regenerating.

7.2 Requirements

Your graph must:

1. **Accept an incoming customer message** as input (a string).
2. **Classify urgency** (`"low"`, `"medium"`, `"high"`) using the LLM, stored in state.
3. **Draft a reply** using the LLM, taking the classified urgency into account (e.g., high-urgency replies should sound more immediate).
4. **Use a static breakpoint** (`interruptAfter`) on the drafting node, so a human always reviews the draft after it's generated, before any further logic runs (practicing Topic 2).
5. **Use a dynamic** `interrupt()` inside a dedicated `humanReviewNode` to collect the human's decision and present three options: `approve`, `edit` (with replacement text), or `reject_regenerate` (with feedback) (practicing Topics 1, 4, and 6).
6. **Route conditionally** based on the decision:
7. `approve` → go straight to a `sendReplyNode`.

8. `edit` → use the edited text as the final reply, then go to `sendReplyNode`.
9. `reject_regenerate` → go back to the drafting node with the feedback incorporated, and repeat (practicing Topic 4's loop pattern).
10. **Cap regenerations at 2 attempts** — if exceeded, automatically escalate to a `humanWritesManuallyNode` that pauses and asks the human to simply provide the full final reply text themselves via `interrupt()`, no more AI attempts.
11. Use a **persistent checkpointer** (`SqliteSaver`), proving (as in Topic 3) that this could survive a real server restart.
12. Print clear console output at every stage so you can trace exactly what happened.

7.3 Suggested State Schema

```
const State = Annotation.Root({
  customerMessage: Annotation<string>({ reducer: (_o, n) => n, default: () => "" }),
  urgency: Annotation<"low" | "medium" | "high">({ reducer: (_o, n) => n, default: () => "low" }),
  draftReply: Annotation<string>({ reducer: (_o, n) => n, default: () => "" }),
  finalReply: Annotation<string>({ reducer: (_o, n) => n, default: () => "" }),
  regenerationCount: Annotation<number>({ reducer: (_o, n) => n, default: () => 0 }),
  regenerationFeedback: Annotation<string>({ reducer: (_o, n) => n, default: () => "" }),
  status: Annotation<"pending" | "approved" | "sent" | "rejected">({
    reducer: (_o, n) => n,
    default: () => "pending",
  }),
});
```

7.4 Suggested Node/Graph Skeleton

```
// classifyUrgencyNode -> draftReplyNode --(interruptAfter)--> humanReviewNode
// humanReviewNode routes to:
//   "approve"          -> sendReplyNode
//   "edit"             -> sendReplyNode (using edited text as finalReply)
//   "reject_regenerate" -> draftReplyNode (if regenerationCount < 2)
//   "humanWritesManually" -> humanWritesManuallyNode (if regenerationCount >= 2)
// humanWritesManuallyNode -> sendReplyNode
// sendReplyNode -> END

const graph = new StateGraph(State)
  .addNode("classifyUrgency", classifyUrgencyNode)
  .addNode("draftReply", draftReplyNode)
  .addNode("humanReview", humanReviewNode)
  .addNode("humanWritesManually", humanWritesManuallyNode)
  .addNode("sendReply", sendReplyNode)
  .addEdge(START, "classifyUrgency")
  .addEdge("classifyUrgency", "draftReply")
  .addEdge("draftReply", "humanReview")
  .addConditionalEdges("humanReview", routeAfterReview, {
    sendReply: "sendReply",
    draftReply: "draftReply",
    humanWritesManually: "humanWritesManually",
  })
  .addEdge("humanWritesManually", "sendReply")
  .addEdge("sendReply", END)
  .compile({
    checkpointer: SqliteSaver.fromConnString("email-assistant.db"),
    interruptAfter: ["draftReply"],
  });
```

Note: Since `interruptAfter: ["draftReply"]` already pauses the graph right after drafting, your `humanReviewNode`'s `interrupt()` call effectively becomes the very first thing that runs *after* you resume from that static breakpoint. This is intentional — it demonstrates that static breakpoints and dynamic `interrupt()`s can be layered together in the same graph, each serving a distinct purpose (a guaranteed pause point vs. a structured question/answer exchange).

7.5 Build Checklist (work through in order)

- [] Step 1: Implement `classifyUrgencyNode` using a simple LLM call that returns one of `"low" | "medium" | "high"`.
- [] Step 2: Implement `draftReplyNode`, incorporating `state.urgency` and (if present) `state.regenerationFeedback` into the prompt, similar to Topic 4's `proposeRefundNode`.
- [] Step 3: Implement `humanReviewNode` with a single `interrupt()` call presenting `draftReply` and accepting `{ decision: "approve" | "edit" | "reject_regenerate", editedText?: string, feedback?: string }`.
- [] Step 4: Implement `routeAfterReview` to branch on `decision`, incrementing `regenerationCount` when looping back, and forcing the escalation path once `regenerationCount >= 2`.
- [] Step 5: Implement `humanWritesManuallyNode` with its own `interrupt()` asking for fully manual reply text.
- [] Step 6: Implement `sendReplyNode` to print the final reply as "sent."
- [] Step 7: Test all four end-to-end paths manually (approve / edit / regenerate-then-approve / regenerate-twice-then-manual) using distinct `thread_id`s, verifying with `getState()` at each pause.
- [] Step 8: (Stretch goal) Split the pause and resume calls into two separate `.ts` files/processes, like Topic 3's lab, to prove this survives a "restart."

7.6 Self-Check Questions Before You Consider It Done

1. If you call `getState()` right after the static `interruptAfter` breakpoint fires, does `snapshot.next` show `humanReview`? If not, why might that be?
2. Does resuming with `decision: "edit"` correctly bypass the AI's `draftReply` entirely and use only the human's text in `sendReplyNode`?
3. After two regenerations, does the graph definitely *stop* asking the AI to try again, and definitely route to `humanWritesManuallyNode` instead?
4. If you kill your terminal mid-pause and restart, does resuming with the same `thread_id` still work correctly (assuming you used `SqliteSaver`)?

8. Phase 4 Master Knowledge Check

Summary of the entire phase

Human-in-the-Loop lets a LangGraph application safely pause before or after sensitive actions, hand control to a human, and resume exactly where it left off — powered entirely by the checkpointers' ability to persist state. There are two pause mechanisms (`interrupt()` for dynamic, data-driven pauses; `interruptBefore` / `interruptAfter` for unconditional, structural pauses), one resume mechanism (`Command({ resume: ... })`), and two ways for a human to directly influence content (returning corrected values through `resume`, or calling `updateState()` independently). Real-world approval workflows combine these primitives with conditional edges to support approve / reject-and-abort / reject-and-revise outcomes, often with a safeguard against infinite revision loops.

Key takeaways across the whole phase

- **No checkpointer, no HITL** — this is true for every single feature in this phase.
- `interrupt()` = ask a question *conditionally*, from inside node code, only when your logic decides it's needed.

- `interruptBefore / interruptAfter` = pause *unconditionally*, configured outside node code, at compile time.
- `Command({ resume: value })` is the dedicated way to answer a pending `interrupt()`; `graph.invoke(null, config)` is enough to continue past a static breakpoint with no new data.
- Code before an `interrupt()` re-runs on every resume — keep it side-effect-free and deterministic.
- Persistent checkpoints (`SqliteSaver` , `PostgresSaver`) let pauses survive process restarts and long delays; `MemorySaver` does not.
- A robust approval workflow supports at least three outcomes (approve / reject-abort / reject-revise) and should guard against unbounded revision loops.
- Human corrections must be explicitly read and applied by your node logic — nothing happens "automatically."

Comprehensive practice questions

1. Explain, in your own words, why `interrupt()` requires the node function to "re-run from the top" on resume, and what risk that creates.
2. Design a routing table (in words) for a content-moderation graph with four possible human decisions: `approve` , `reject` , `escalate_to_legal` , `request_more_context` . Which existing topic's pattern would you reuse to build this?
3. You need a workflow where *every* payment over \$1,000 always requires approval, but payments under \$1,000 should never pause. Would you implement this with `interruptBefore` alone, `interrupt()` alone, or a combination? Justify your answer.
4. A teammate says: "We don't need a persistent checkpoint, `MemorySaver` is fine, because our approvals always happen within the same `invoke()` call." Is this teammate's reasoning correct? Why or why not?
5. What's the smallest possible change you could make to Topic 4's lab to add a hard cap on revisions, and where exactly would that check belong (which function)?

Capstone exercise

Sketch (diagram or written description) a **HITL-gated database migration tool**: an agent that proposes a SQL migration script for a database, requires human approval before running it, allows the human to edit the SQL directly, and automatically rolls back any change if a post-run health check (a separate node) fails — feeding that failure back as `reviewerFeedback` -style context for a possible second human-reviewed attempt. Identify exactly which Phase 4 topic each piece of this design draws from.

9. Glossary (Phase 4 terms)

Term	Definition
Human-in-the-Loop (HITL)	A design pattern where a human reviews, approves, edits, or rejects an AI agent's action before (or after) it takes effect.
<code>interrupt()</code>	A function called inside a node to dynamically and conditionally pause graph execution, sending a payload out and later receiving a resume value back at that exact point.
Static breakpoint	An unconditional pause configured at <code>.compile()</code> time via <code>interruptBefore / interruptAfter</code> , requiring no code changes inside the node.
<code>interruptBefore</code>	Compile option causing the graph to pause immediately before a named node runs.
<code>interruptAfter</code>	Compile option causing the graph to pause immediately after a named node finishes.
	An object used to control a graph's continuation; in HITL contexts, primarily used as

Command	<code>Command({ resume: value })</code> to answer a pending <code>interrupt()</code> .
Resume value	The data passed via <code>Command({ resume: ... })</code> that an <code>interrupt()</code> call effectively "returns" once execution continues.
Approval workflow	A pattern combining <code>interrupt()</code> /breakpoints with conditional edges to support approve / reject-abort / reject-revise outcomes.
<code>updateState()</code>	A method (introduced in Phase 3) that directly overwrites a paused thread's state without going through any node — useful for out-of-band human corrections.
<code>getState()</code>	A method to inspect a thread's current checkpoint, including <code>values</code> (current state) and <code>next</code> (which node(s) will run next).
Persistent checkpointer	A checkpointer backend (e.g., <code>SqliteSaver</code> , <code>PostgresSaver</code>) that survives process restarts, as opposed to <code>MemorySaver</code> .
<code>thread_id</code>	A unique identifier passed in <code>config.configurable</code> that links a paused run to the eventual resume call.

End of Phase 4 — Human-in-the-Loop. Next phase: Multi-Agent & Tools (Subgraphs, Supervisor/Network/Hierarchical/Swarm architectures, ToolNode, and prebuilt agents).